Structural and Semantic Verification for Reliable Text-to-BPMN Process Modeling

Author Information Omitted for Review

Affiliation Omitted for Review

Abstract. Process modeling is the practice of creating a visual, graphical representation of a business process model in standardized languages such as Business Process Model and Notation (BPMN). Many such models, however, are still derived manually from textual procedures, policies, and requirements. Large Language Models (LLMs) can extract process semantics from text. However, direct text-to-BPMN generation often produces structurally invalid models or models with semantic drift. To address this issue, we present MAC (Multi-Agent Collaboration), a gated framework that treats text-to-BPMN modeling as a sequential structural-semantic admissibility problem. MAC decomposes text-to-BPMN modeling into typed semantic extraction, BPMN graph construction, structural admissibility checking, source-grounded semantic diagnosis, and localized repair. The framework first converts text into a typed intermediate representation, then maps it to a BPMN control-flow graph, and subsequently checks graph admissibility with deterministic structural constraints followed by source-grounded semantic diagnosis. These two verification gates produce typed diagnostics localized to affected graph or semantic symptoms, guiding repair toward the final model without regenerating the entire model. In the PET benchmark, MAC further improved activity recovery, role consistency, relation recovery, semantic pass rate, and structural soundness under the same evaluation protocol, outperforming existing RAP and ProMoAI baseline models.Additional supplementary material is available at `https://maipdf.cn/file/at6a5d80595aec0/pdf`.

Keywords: business process modeling · BPMN · multi-agent systems · semantic verification

1 Introduction

Service-oriented computing organizes software capabilities as discoverable, composable, and governable services. Business process modeling, in this setting, defines service interactions, role responsibilities, exception paths, and constraints in standardized representations, so that stakeholders from executives to IT developers can explore how tasks, data, and people interact in the same picture. Over the past decades, Business Process Model and Notation (BPMN) has remained a common representation of process models because it effectively combines human-readable process diagrams with executable control-flow structure [1]. Yet many

service processes are still specified as natural-language procedures, requirements, policies, or standard operating documents. Turning these descriptions into reliable BPMN models is therefore important for downstream tasks, such as service automation and compliance analysis.

Recent Large Language Models (LLMs) provide a promising interface for this text-to-BPMN conversion because they can parse long and noisy procedural text [2]. However, current LLM-based studies reveal that direct text-to-BPMN generation remains unreliable, with outputs possibly containing isolated nodes, deadlocks, dangling branches, or incorrect relations [3]. In fact, a generated BPMN model must satisfy strict structural constraints such as unique start and end behavior, reachability, and branch closure [4]. In addition, it must preserve source-grounded activities, actor assignments, branch conditions, and inter-activity relations.

The central difficulty is that process modeling must effectively combine semantic interpretation with formal artifact construction. The current end-to-end approach usually identifies activities, normalizes labels, infers actors, recovers control flow, serializes BPMN, and checks structural validity with a single prompt. In this case, local semantic plausibility and global graph validity can conflict. A text-faithful response may still produce dangling branches or unreachable fragments, while a structurally regular graph may omit source activities, misassign actors, or distort branch relations. We therefore treat text-to-BPMN modeling as a staged admissibility problem: *a candidate should first become structurally admissible before semantic consistency is diagnosed*, and *repairs should target localized faults rather than regenerate the whole model*.

Motivated by this insight, we propose MAC, a multi-stage framework for reliable text-to-BPMN modeling. MAC decomposes the task into a pipeline of semantic extraction, structural mapping, structural verification, semantic verification, and feedback-driven repair. The framework first constructs a typed representation of the source process, maps it to a BPMN control-flow graph, validates graph admissibility, and then diagnoses source-grounded semantic mismatches only after the graph is structurally usable. Typed diagnostics from the verification stages are used to guide localized repair of affected nodes, flows, roles, or relations while preserving unaffected model components.

In addition, our design separates runtime repair signals from offline benchmark scoring. During generation, MAC uses only the source description, the intermediate representation, and the current BPMN candidate; the labeled annotations are used only after generation to compute evaluation metrics. This separation avoids benchmark leakage and clarifies the claim: MAC studies whether verification-guided repair improves reliability under a fixed automatic protocol, not whether LLMs replace human process analysts in all BPMN settings.

In summary, our work makes three contributions. First, we formulate text-to-BPMN modeling as a sequential dual-gated admissibility problem for service-process modeling from natural language. Second, we design a verification-repair mechanism that separates graph-level BPMN admissibility from source-grounded semantic consistency. Third, we evaluate MAC on PET with controlled baselines,

coarse-grained module ablations, compact fine-grained ablations on feedback granularity and intermediate-representation structure, with additional seeded-error and Petri-net consistency checks reported in Supplementary Tables S5 and S6. These results show that explicit structural-semantic gating can improve the reliability of LLM-generated BPMN artifacts under a controlled text-to-BPMN evaluation protocol.

2 Related Work

Process extraction from text. Business process extraction has been long studied, which asims to identify activities, actors, and control-flow cues from natural-language descriptions. Early rule-based and semantic-parsing approaches established the feasibility of deriving process models from text [5]. Later datasets, especially PET, enabled more comparable evaluation by providing expert annotations for activities, roles, gateways, and relations [6]. These benchmarks also show a persistent gap between extracting local elements and constructing structurally usable process models. The work on parallelism annotation, process-model similarity, and workflow verification further indicates that activity-label overlap alone cannot capture structural or behavioral adequacy [7–11]. MAC follows this lesson by evaluating generated BPMN candidates through activity recovery, role consistency, relation consistency, and verifier-defined structural soundness.

LLM-based process modeling. LLMs have shifted text-to-process modeling from rule engineering toward flexible semantic interpretation. ProMoAI shows that generative AI can support process modeling and iterative refinement [3]. BPMN Assistant highlights the value of intermediate JSON-like representations for BPMN creation and editing [12]. Multi-stage text-to-BPMN generation further shows that validation and repair can improve generation reliability [13]. MAC builds on this direction but uses a stricter verification-repair protocol: it performs semantic diagnosis only after structural admissibility is satisfied. Moreover, it guides repair with typed diagnostics rather than open-ended regeneration.

Agentic reasoning and repair. Multi-agent LLM systems decompose complex tasks across specialized roles. MetaGPT and SWE-agent illustrate role specialization and explicit interfaces [14, 15]; workflow-oriented agents emphasize dynamic control and feedback-aware coordination [16]. Reflexion, Self-Refine, chain-of-thought, Tree of Thoughts, ReAct, RAP, and LATS provide related mechanisms for decomposition, search, and feedback-driven revision [17–23]. MAC adapts these ideas to a formal modeling setting in which feedback is grounded in executable BPMN artifacts: graph parsing, reachability, branch closure, and source-grounded role or relation mismatches.

The closest connection to MAC is therefore feedback grounded in an executable artifact. In software agents, compiler errors and unit tests localize failures; in BPMN generation, the analogous signals are graph parsing, reachability, branch closure, and source-grounded role or relation mismatches. MAC treats

these signals as first-class coordination objects rather than as post hoc evaluation summaries. This distinction is important because a BPMN candidate is not merely a text response: it is a formal object whose local edits can change global behavior.

3 Problem Definition and Verification Scope

3.1 Task Setting

Given a natural-language service-process description T, the goal of our task is to generate a BPMN process model G that preserves source-grounded activities, roles, branch conditions, and control-flow relations in T. Here, $G = (V, E, L)$, where V contains a start node v_{start}, an end node v_{end}, activity nodes V_{act}, and gateway nodes V_{gtw}, or $V = \{v_{\text{start}}\} \cup \{v_{\text{end}}\} \cup V_{\text{act}} \cup V_{\text{gtw}}$, while the edge set $E \subseteq V \times V$ represents directed sequence flows (v_i, v_j) in which $v_i, v_j \in V$. The label set L assigns semantic labels to nodes in V and edges in E, such as activity names, actor assignments, gateway types, and branch conditions.

3.2 Structural Verification

A generated graph G is structurally admissible if it satisfies three graph-level structural constraints. First, **topological completeness** requires every non-start node to have an incoming edge and every non-end node to have an outgoing edge:

$$\forall v \in V \setminus \{v_{\text{start}}\},\ \text{in-degree}(v) \geq 1, \tag{1}$$

$$\forall v \in V \setminus \{v_{\text{end}}\},\ \text{out-degree}(v) \geq 1. \tag{2}$$

Second, **global reachability** requires each intermediate node to lie on at least one path from the start event to the end event. Let $v_i \rightsquigarrow v_j$ represent that $(v_i, v_j) \in E^+$, where E^+ denotes the transitive closure of E. The constraint of global reachability can be defined as:

$$\forall v_i \in V \setminus \{v_{\text{start}}, v_{\text{end}}\} : v_{\text{start}} \rightsquigarrow v_i,\ v_i \rightsquigarrow v_{\text{end}}. \tag{3}$$

Third, **branch closure** requires every split gateway in the modeled acyclic branch structure to reconverge before termination. For gateway g, let $\text{Pred}(g) = \{v \mid (v, g) \in E\}$ and $\text{Succ}(g) = \{v \mid (g, v) \in E\}$. A gateway with $|\text{Succ}(g)| \geq 2$ is a split, and a gateway with $|\text{Pred}(g)| \geq 2$ is a join. The constraint of branch closure can be defined as:

$$\begin{aligned} &\forall g_{\text{split}} \in V_{\text{gtw}},\ |\text{Succ}(g_{\text{split}})| \geq 2 \\ &\Rightarrow \exists g_{\text{join}} \in V_{\text{gtw}}, |\text{Pred}(g_{\text{join}})| \geq 2 \wedge (\forall p \in \text{Paths}(g_{\text{split}}, v_{\text{end}}) : g_{\text{join}} \in p). \end{aligned} \tag{4}$$

3.3 Semantic Verification

As the second verification gate, semantic verification checks whether a structurally usable BPMN graph preserves the source-side activities, role assignments, and control-flow relations. Let $\mathcal{A}_T$, $\mathcal{R}_T$, and $\mathcal{E}_T$ be the activity set, role-assignment set, and typed relation-flow set extracted from the source text T, the generated graph G yields the corresponding sets $\mathcal{A}_G$, $\mathcal{R}_G$, and $\mathcal{E}_G$.

Activity match is based on label similarity. For a source activity $a \in \mathcal{A}_T$ and a generated activity $\hat{a} \in \mathcal{A}_G$, let $s(a, \hat{a}) \in [0, 1]$ be their normalized textual similarity. Given the activity threshold τ_a, the basic matching relation is

$$a \sim \hat{a} \quad \Leftrightarrow \quad s(a, \hat{a}) \geq \tau_a. \tag{5}$$

Here, a and $\hat{a}$ denote the source and generated activity labels respective, $s(\cdot, \cdot)$ represents the similarity function, and τ_a is the minimum similarity required to regard two activities as the same process step.

Based on this activity match, semantic verification checks three compact conditions:

$$\begin{aligned} C_{\text{act}} &: \forall a \in \mathcal{A}_T,\ \exists \hat{a} \in \mathcal{A}_G : a \sim \hat{a}, \\ C_{\text{role}} &: \forall (a, r) \in \mathcal{R}_T,\ \exists (\hat{a}, \hat{r}) \in \mathcal{R}_G : a \sim \hat{a} \wedge r = \hat{r}, \\ C_{\text{flow}} &: \forall (a_i, a_j, \rho, c) \in \mathcal{E}_T,\ \exists (\hat{a}_i, \hat{a}_j, \hat{\rho}, \hat{c}) \in \mathcal{E}_G : \\ &\qquad a_i \sim \hat{a}_i \wedge a_j \sim \hat{a}_j \wedge \rho = \hat{\rho} \wedge c = \hat{c}. \end{aligned} \tag{6}$$

Eq. (6) defines r and $\hat{r}$ as source and generated roles, ρ and $\hat{\rho}$ as source and generated relation-flow types (sequence, choice, or parallel relation), and c and $\hat{c}$ as source and generated branch conditions with c defaulting to $\varnothing$ when no explicit condition is specified. *AgentSem* reports the failed condition and the affected activity, role, or relation-flow tuple so that *AgentFix* can repair the localized semantic mismatch later.

4 The MAC Framework

4.1 Overall Architecture

MAC contains five specialized agents, as shown in Fig. 1. First, AgentExtract transforms the source text into a typed intermediate representation (IR), followed by AgentModel, which serializes the IR into the BPMN XML file. Afterwards, AgentVer and AgentSem check structural admissibility and source-grounded semantic consistency, respectively. Finally, AgentFix repairs localized structural or semantic faults and returns the repaired model to AgentVer.

The architecture adheres to two progressive coordination principles: (1) representational formalization escalates along the pipeline, as unstructured text is mapped to a typed IR, serialized to BPMN XML, and ultimately parsed into a verification graph; (2) feedback resolution localizes faults, whereby structural faults pinpoint graph breakpoints, semantic errors trace to source-grounded

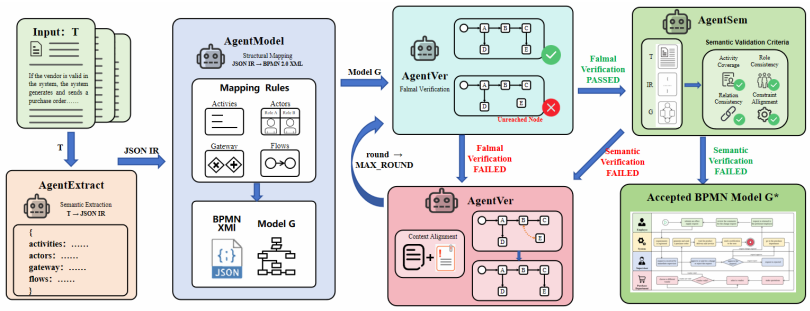

Fig. 1. Overview of MAC. It first extracts a typed intermediate representation and maps it to BPMN, then checks structural admissibility and diagnoses source-grounded semantic consistency, and finally repairs only localized faults.

mismatches, and repair operations remain constrained to the reported anomaly whenever feasible.

AgentExtract. The agent *AgentExtract* translates procedural text into a JSON-structured intermediate representation (IR) that normalizes activities as atomic actions, assigns actors to swimlanes, encodes gateways as XOR/AND branches, and defines flow precedence. Discrete fields for actors, labels, types, and conditions enable diagnostic localization of missing elements, role errors, and relational faults. Serving as a traceable contract between natural-language understanding and BPMN serialization, this compartmentalization allows MAC to discriminate among omissions, swimlane misassignments, and connectivity errors. Furthermore, this structured representation facilitates localized semantic repair: *AgentFix* can update implicated role assignments or routing paths while preserving unaffected parts of the process description.

AgentModel. The agent *AgentModel* translates the IR into a BPMN XML file within a predefined model subset, mapping activities, actors, decision branches, and directed relations to task nodes, swimlanes, typed gateways, and sequence flows, respectively. The conversion enforces singleton start/end events, explicit edge directionality, and branch-condition preservation while precluding constructs outside the model subset, thereby maintaining tractable verification and benchmark alignment.

AgentVer. The agent *AgentVer* is a deterministic structural gate rather than a generative judge. It parses the given BPMN XML file, derives the sequence-flow graph, and checks the constraints defined in Section 3.2. The generated report localizes the violation type and affected nodes or flows. The report is intentionally operational: a candidate that fails *AgentVer* is not passed to later semantic checking, because semantic diagnosis over an unparsable or structurally invalid graph can hide unresolved control-flow faults. Instead of returning only that a model is invalid, *AgentVer* identifies the constraint family and the affected graph element when possible, which enables repairs such as adding a missing outgoing sequence flow, reconnecting an unreachable fragment, or pairing a split

gateway with a join. These edits are sufficient for the failure classes targeted by the modeled BPMN subset.

AgentSem. The agent *AgentSem* accepts a candidate passed by *AgentVer* and identifies missing activities, incorrect roles, incorrect relations, and branch-condition mismatches. The output is a machine-readable diagnostic report used by *AgentFix*. This runtime check is separate from offline benchmark scoring: the annotations never enter the repair loop, while the offline evaluator later reports activity, role, relation, and semantic pass metrics.

AgentFix. The agent *AgentFix* performs localized in-context repair without updating model parameters. Structural diagnostics trigger topology edits, such as reconnecting an unreachable fragment or closing an unmerged branch, while semantic diagnostics trigger edits to activity labels, roles, branch conditions, or relation routing. The repair prompt requires unaffected model elements to remain unchanged whenever possible, and every repaired model returns to *AgentVer* because semantic edits can break graph validity. This locality is the main difference between MAC and unconstrained self-refinement. Whole-model regeneration can remove one violation while silently changing correct activities, roles, or branches. MAC instead treats a candidate as a partially valid artifact: each repair round should preserve already accepted components unless the diagnostic explicitly implicates them. Although this design does not guarantee complete semantic adequacy, it makes the repair trajectory auditable.

4.2 Pipeline Integration

Table 1 summarizes the whole pipeline. The key design choice lies in the ordering of the gates: structure first, semantics second, and repair after localized diagnosis.

MAC differs from one-shot generation and unconstrained self-refinement because it does not regenerate the whole process after every failure. It first identifies which structural or semantic condition fails, then repairs only the affected substructure indicated by the diagnostic report. The structural gate is repeated after semantic repair because edits to labels, branches, or relations can invalidate reachability or branch closure. The maximum-round parameter bounds cost and returns the final candidate with the last structural and semantic reports if repair does not converge.

The loop also makes failure observable. If a candidate remains invalid, the final report records whether the bottleneck is topological, such as an unreachable fragment, or semantic, such as a missing role or relation mismatch. This is useful for service-process modeling because an analyst can inspect a bounded repair trace instead of comparing several independently regenerated BPMN files.

Table 1. Gated structural-semantic pipeline of MAC. Structure is checked before source-grounded semantics, and each failed gate returns localized diagnostics for repair.

Input: process description T
Output: BPMN process graph G with verification reports

IR $\leftarrow$ AgentExtract(T)
$G_0 \leftarrow$ AgentModel(IR)
$G \leftarrow G_0$, $round \leftarrow 0$
while $round <$ MAX_ROUNDS do
  $R_{\text{struct}} \leftarrow$ AgentVer(G)
  if $R_{\text{struct}} \neq$ PASSED then
    $G \leftarrow$ AgentFix(G, R_{struct});
    $round \leftarrow round + 1$; continue
  end if
  $R_{\text{sem}} \leftarrow$ AgentSem(T, IR, G)
  if $R_{\text{sem}} =$ PASSED then return G
  $G \leftarrow$ AgentFix(G, R_{sem});
  $round \leftarrow round + 1$
end while
return G with R_{struct} and R_{sem}

5 Experiments

5.1 Experimental Setup

Dataset. We evaluate MAC on PET, a benchmark covering domains such as finance, administration, and healthcare [6]. PET includes linear procedures as well as processes with branching, parallelism, and feedback loops, making it suitable for testing activity recovery and control-flow validity. The annotations in PET support evaluation at both the element level and the graph-relation level, which matches the focus of our work on structural validity and source-grounded semantic preservation.

Evaluation Metrics. For PET evaluation, the source-side sets introduced in Section 3.3 are instantiated as the gold activity set $\mathcal{A}^*$, role-assignment set $\mathcal{R}^*$, and typed relation set $\mathcal{E}^*$. The generated graph yields $\mathcal{A}_G$, $\mathcal{R}_G$, and $\mathcal{E}_G$. An activity pair $(a, \hat{a})$ is matched when $s(a, \hat{a}) \geq \tau_a$, yielding a one-to-one activity matching M_A.

We report the following metrics throughout the experiment:

$$\text{Precision} = \frac{|M_A|}{|\mathcal{A}_G|}, \quad \text{Recall} = \frac{|M_A|}{|\mathcal{A}^*|}, \quad \text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{7}$$

$$\text{ANSS} = \frac{1}{|M_A|} \sum_{(a,\hat{a}) \in M_A} s(a, \hat{a}). \tag{8}$$

For role assignment, let $M_A^{\text{role}} \subseteq M_A$ denote matched activities with gold role annotations, and M_R denote the subset whose generated role agrees with the

gold role. Then, Role accuracy can be defined as:

$$\text{RoleAcc} = \frac{|M_R|}{|M_A^{\text{role}}|}. \tag{9}$$

For typed control-flow relations, let M_E denote the one-to-one matched relation set between the generated typed relations $\mathcal{E}_G$ and the gold typed relations $\mathcal{E}^*$. Thus, relation precision, recall, and F1 can be defined as:

$$P_E = \frac{|M_E|}{|\mathcal{E}_G|}, \quad R_E = \frac{|M_E|}{|\mathcal{E}^*|}, \quad \text{RelationF1} = \frac{2P_E R_E}{P_E + R_E}. \tag{10}$$

The semantic pass indicator combines the three source-grounded semantic criteria used by *AgentSem*. We use $\mathbf{1}\{\cdot\}$ as an indicator function, which equals 1 when the enclosed condition is true and 0 otherwise:

$$\text{Sem.Pass} = \mathbf{1}\{\text{F1} \geq \theta_a \wedge \text{RoleAcc} \geq \theta_r \wedge \text{RelationF1} \geq \theta_e\}, \tag{11}$$

where θ_a, θ_r, and θ_e are the thresholds for activity recovery, role accuracy, and typed-relation recovery, respectively.

Soundness is reported as an indicator that the generated graph passes all structural constraints defined in Section 3.2:

$$\text{Soundness} = \mathbf{1}\{C_{\text{topo}}(G) \wedge C_{\text{reach}}(G) \wedge C_{\text{branch}}(G)\}. \tag{12}$$

Here $C_{\text{topo}}(G)$ checks topological completeness, namely the in-degree and out-degree constraints in Eqs. (1)–(2); $C_{\text{reach}}(G)$ checks global reachability as defined in Eq. (3); and $C_{\text{branch}}(G)$ checks branch closure as defined in Eq. (4). If a denominator is zero in a case-level ratio, the evaluator assigns the corresponding score according to the same bounded $[0, 1]$ convention used in the metric scripts.

For aggregate results, these scores are averaged over generated outputs, with F1, RoleAcc, RelationF1, Sem.Pass, and Soundness computed at the case level before averaging. PET annotations are used only by the offline evaluator after generation and are not used inside *AgentSem* or *AgentFix*.

Baselines. The comparison includes six baselines. **Zero-Shot** directly prompts the LLM to generate BPMN from the source text without demonstrations or reasoning scaffolds. **Few-Shot** adds fixed text-to-BPMN demonstrations reused across all PET cases. **Zero-Shot-CoT** adds stepwise guidance before BPMN generation, asking the model to identify activities, actors, decision points, conditions, and parallel sections. **Few-Shot-CoT** combines the fixed demonstrations with the same stepwise guidance. **RAP-BPM** is a BPM-specific adaptation of RAP [22] and serves as a planning-oriented baseline, while **ProMoAI** follows the process-modeling-with-generative-AI setting of Kourani et al. [3] under our unified parser and metric pipeline.

These baselines establish controlled comparison points for the central design question of this paper: whether explicit structural-semantic gating and localized repair improve text-to-BPMN reliability beyond direct prompting, demonstration-based prompting, reasoning scaffolds, planning-oriented generation, and iterative generative process modeling.

LLM Backbones. To select an appropriate LLM backbone for the main experiments, we first evaluate MAC with three candidate backbones: GPT-4o, Qwen3.5-Plus, and DeepSeek-V3. Table 2 reports this backbone-selection study on the same PET cases, using the same source descriptions, output target, parser, and metric scripts. The three backbones show comparable activity-level trends and the same verifier-defined structural soundness, indicating that MAC is not highly sensitive to the tested backbone choices. We therefore use GPT-4o as the unified backbone because it offers faster inference, broad availability, and a widely used reference point for LLM-based process-modeling evaluation.

Table 2. Backbone check for MAC on PET. All reported values are means computed under an identical evaluation protocol. The comparison is used to assess backbone sensitivity rather than to rank general LLM capability.

Backbone	F1	ANSS	Soundness	Time/sample
GPT-4o	**0.645**	0.462	1.000	**21.39**s
Qwen3.5-Plus	0.612	0.451	1.000	284.59s
DeepSeek-V3	0.603	**0.476**	1.000	52.72s

5.2 Main Results

Table 3 reports ten-run averaged results for the complete PET benchmark, which demonstrates MAC achieves the best scores across activity recovery, soft label similarity, role consistency, relation recovery, semantic pass rate, and verifier-defined structural soundness.

Table 3. Overall comparison on PET under a ten-run evaluation protocol. Values are means across ten independent runs. The best ones are indicated in bold, while the second best ones are underlined.

Method	Prec.	Rec.	F1	ANSS	Role Acc.	Rel. F1	Sem. Pass	Sound.
Zero-Shot	0.542	0.391	0.408	0.306	0.281	0.206	0.378	0.693
Few-Shot	0.642	0.421	0.498	0.356	0.330	0.310	0.521	0.993
Zero-Shot-CoT	0.576	0.303	0.476	0.306	0.308	0.315	0.511	0.867
Few-Shot-CoT	0.601	0.371	0.503	0.349	0.425	0.410	0.556	0.889
RAP-BPM [22]	<u>0.643</u>	0.378	<u>0.521</u>	<u>0.365</u>	<u>0.496</u>	<u>0.471</u>	<u>0.880</u>	**1.000**
ProMoAI [3]	0.517	<u>0.454</u>	0.429	0.292	0.465	0.425	0.785	<u>0.911</u>
MAC (Ours)	**0.689**	**0.538**	**0.639**	**0.537**	**0.784**	**0.632**	**0.933**	**1.000**

MAC improves activity F1 from 0.521 to 0.639 over the strongest F1 baseline, RAP-BPM. The improvement is significant on semantic consistency metrics:

RoleAcc rises from 0.496 to 0.784, RelationF1 from 0.471 to 0.632, and Sem.Pass from 0.880 to 0.933. Recall also improves without sacrificing precision, suggesting that the repair loop recovers source-grounded elements rather than simply adding nodes.

The recall gain is important for procedural text because missing branch-specific actions can change the meaning of a service process even when the remaining activities are precise. At the same time, the precision gain suggests that MAC does not simply add more activities to improve recall. The combination of improved activity recovery and stronger role/relation metrics indicates that the dual-gate loop contributes to both coverage and source-grounded consistency.

Structural compliance shows a complementary pattern. MAC and RAP-BPM both reach Soundness = 1.000, while ProMoAI and prompt-only baselines leave structural violations under the same verifier. The difference between RAP-BPM and MAC suggests that structural soundness alone is not the full bottleneck: the second gate matters because it directs repair toward source-grounded role and relation mismatches that are invisible to structural checks.

To further examine verifier-defined structural reliability, Supplementary Tables S5 and S6 report a seeded-error audit of *AgentVer* and a Petri-net consistency check over the generated MAC outputs.

This pattern also explains why the framework is evaluated with both graph-level and source-grounded metrics. A process model that is structurally sound but assigns tasks to the wrong participant may still misrepresent service responsibility, while a semantically close but structurally invalid model cannot be reliably executed or analyzed. MAC is designed for the intersection of these requirements rather than optimizing either one in isolation.

5.3 Ablation Studies

Table 4 isolates the contribution of MAC's main components by comparing ablated variants with the full MAC result.

Table 4. Coarse-grained module ablation under the PET evaluation protocol

Variant	F1	ANSS	Soundness	RoleAcc	RelationF1	Sem.Pass
Single Agent	0.347	0.190	0.822	0.271	0.224	0.000
w/o Extraction	0.522	0.331	<u>0.978</u>	0.331	0.284	0.000
w/o Structural Gate	<u>0.644</u>	<u>0.422</u>	0.889	<u>0.601</u>	0.402	0.511
w/o Refinement	0.643	0.420	0.911	0.591	<u>0.419</u>	<u>0.578</u>
w/o Semantic Gate	0.568	0.407	1.000	0.572	0.342	0.044
MAC (full model)	**0.639**	**0.537**	**1.000**	**0.784**	**0.632**	**0.933**

In this component analysis, the single-agent variant performs poorly, indicating that the combined task is difficult to handle as a single undifferentiated generation step under the same protocol. Removing the extraction stage reduces

activity-level and soft semantic similarity, indicating that the typed IR stabilizes downstream mapping and diagnosis.

The structural and semantic gates address different failure modes. Without the structural gate, activity F1 remains high (0.6440), but Soundness drops to 0.8889 and Sem.Pass to 0.5111; without the semantic gate, Soundness remains 1.0000, but Sem.Pass falls to 0.0444 and RelationF1 to 0.3415. Thus, structural validity is necessary for graph usability, while semantic diagnosis is necessary for source-grounded role and relation consistency.

The ablation also shows that extraction, checking, and repair are not interchangeable. Removing AgentExtract weakens the typed contract that later agents rely on, while removing refinement leaves diagnostics unused. The relatively high activity F1 of some incomplete variants is therefore misleading: they can recover labels but fail to turn those labels into a usable and source-grounded BPMN artifact.

The w/o Refinement row further separates diagnosis from repair. A pipeline may know that a candidate is invalid or semantically inconsistent, but without a targeted repair step those reports do not necessarily become better BPMN artifacts. Conversely, the w/o Semantic Gate row shows that structural repair alone can produce well-formed graphs whose roles and relations remain inconsistent with the source. The full model combines both forms of feedback: structural diagnostics keep the graph usable, while semantic diagnostics steer repair toward source-grounded meaning. The following compact fine-grained ablations further examine two internal design decisions: verifier-feedback granularity and intermediate-representation structure.

Table 5. Compact fine-grained ablations on verifier-feedback granularity and intermediate-representation structure. Values are single-run component-analysis results under the same PET evaluation protocol as the coarse-grained ablation. The table reports the common core metrics used across the main and ablation comparisons.

Study	Variant	F1	ANSS	Soundness
Feedback	Boolean-Only	0.504	0.375	1.000
Feedback	Node-ID	0.497	0.369	1.000
Feedback	Error-Type	0.521	0.403	1.000
Feedback	Full NL	**0.604**	**0.527**	**1.000**
IR	Direct-Gen	0.428	0.290	0.000
IR	Flat-JSON	0.434	0.321	1.000
IR	Partial-IR	0.388	0.417	1.000
IR	Full IR	**0.615**	**0.509**	**1.000**

Table 5 reports the same core metrics used to compare the main methods and coarse-grained variants. The verifier-repair loop is robust to several feedback formats: Boolean-only, node-localized, error-typed, and full natural-language diagnostics all reach verifier-defined Soundness = 1.0000. Full natural-language

feedback obtains the highest F1 and ANSS among the feedback variants, but this single-run ablation should be interpreted conservatively: richer diagnostics improve repair informativeness in this setting, while the structural admissibility result does not require verbose feedback. The IR ablation shows a clearer structural effect. Direct generation is structurally unsound in this setting, whereas structured IR variants recover verifier-defined Soundness = 1.0000. Full IR obtains the strongest F1 and ANSS among the tested IR variants, supporting the use of typed activities, actors, gateways, and flow relations as a contract between extraction, modeling, verification, and repair.

6 Discussion

Implications. The results suggest that text-to-BPMN generation should be evaluated as both a graph-construction problem and a source-grounding problem. The gains of MAC come from enforcing these two requirements at different stages: structural checking makes the BPMN artifact usable as a process graph, while semantic checking tests whether the usable graph preserves activities, roles, branch conditions, and relations from the source text. The ablation results further show that typed intermediate representations and localized diagnostics provide useful contracts between generation, verification, and repair.

Threats to validity. The main threat to external validity concerns the evaluation scope. PET provides expert-annotated process descriptions with activities, roles, gateways, and relations, which fits the text-to-BPMN setting in our work. Nevertheless, deployment on longer industrial SOPs or event-rich BPMN models would require additional validation. The structural checker defines graph-level admissibility for the control-flow constructs evaluated in this study; extending it to full BPMN execution semantics is a natural next step. Because semantic adequacy can involve domain-specific intent beyond the PET annotations, human review remains a complementary validation step for deployment settings.

The reported numbers may also vary with the LLM backbone, prompt examples, parser behavior, and label-normalization threshold. We reduce this risk by using the same source descriptions, parser, metric scripts, and evaluation protocol across methods, and by averaging the main comparison over ten runs. The RAP-BPM and ProMoAI baselines are evaluated under this unified protocol for comparability.

7 Conclusion

In this paper, we presented MAC, a gated structural-semantic multi-agent framework for text-to-BPMN process modeling. MAC separates semantic extraction, BPMN mapping, structural verification, semantic verification, and localized repair. Its core insight lies in that a generated process model should first become structurally admissible before source-grounded semantic consistency is diagnosed

and repaired. The extensive experiment demonstrates that MAC improves activity recovery, role consistency, relation recovery, semantic pass rate, and verifier-defined soundness under a unified evaluation protocol, with ablations indicating that both module-level gates and fine-grained design choices affect the final structure-semantics trade-off. Future work should extend MAC to richer BPMN constructs, broader service-process datasets, human semantic evaluation, and adaptive routing policies that reduce cost without removing verification gates.

References

1. Object Management Group: Business process model and notation (BPMN), version 2.0.2. OMG formal specification formal/2013-12-09 (2014), https://www.omg.org/spec/BPMN/2.0.2
2. Brown, T.B., Mann, B., Ryder, N., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems 33 (NeurIPS 2020). pp. 1877–1901 (2020), https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html
3. Kourani, H., Berti, A., Schuster, D., van der Aalst, W.M.P.: ProMoAI: Process modeling with generative AI. In: Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI 2024). pp. 8708–8712 (2024). https://doi.org/10.24963/ijcai.2024/1014, https://www.ijcai.org/proceedings/2024/1014
4. Mendling, J., Reijers, H.A., van der Aalst, W.M.P.: Seven process modeling guidelines (7pmg). Information and Software Technology **52**(2), 127–136 (2010). https://doi.org/10.1016/j.infsof.2009.08.004
5. Friedrich, F., Mendling, J., Puhlmann, F.: Process model generation from natural language text. In: CAiSE 2011. LNCS, vol. 6741, pp. 482–496. Springer (2011). https://doi.org/10.1007/978-3-642-21640-4_36
6. Bellan, P., van der Aa, H., Dragoni, M., Ghidini, C., Ponzetto, S.P.: PET: An annotated dataset for process extraction from natural language text. In: BPM 2022 Workshops. pp. 315–321. Springer (2022). https://doi.org/10.1007/978-3-031-25383-6_23
7. Lê, P.N., Schneider-Depré, C., Goossens, A., Stevens, A., Leribaux, A., De Smedt, J.: Leveraging machine learning and enhanced parallelism detection for BPMN model generation from text (2025). https://doi.org/10.48550/arXiv.2507.08362, https://arxiv.org/abs/2507.08362
8. Dijkman, R., Dumas, M., van Dongen, B., Käärik, R., Mendling, J.: Similarity of business process models: Metrics and evaluation. Information Systems **36**(2), 498–516 (2011). https://doi.org/10.1016/j.is.2010.09.006, https://doi.org/10.1016/j.is.2010.09.006
9. van der Aalst, W.M.P.: Workflow verification: Finding control-flow errors using petri-net-based techniques. In: Business Process Management. pp. 161–183. LNCS, Springer (2000). https://doi.org/10.1007/3-540-45594-9_11
10. van der Aalst, W.M.P., ter Hofstede, A.H.M., Kiepuszewski, B., Barros, A.P.: Workflow patterns. Distributed and Parallel Databases **14**(1), 5–51 (2003). https://doi.org/10.1023/A:1022883727209
11. van der Aalst, W.M.P.: The application of petri nets to workflow management. Journal of Circuits, Systems and Computers **8**(1), 21–66 (1998). https://doi.org/10.1142/S0218126698000043

12. Licardo, J.T., Tankovic, N., Etinger, D.: BPMN assistant: An LLM-based approach to business process modeling (2026). https://doi.org/10.48550/arXiv.2509.24592
13. Matei, I., Zhenirovskyy, M., Menaka Sekar, P.K., Wong, H.Y.: Automated BPMN model generation from textual process descriptions: A multi-stage LLM-driven approach (2026). https://doi.org/10.48550/arXiv.2604.12105
14. Hong, S., Zhuge, M., Chen, J., et al.: MetaGPT: Meta programming for a multi-agent collaborative framework. In: Proceedings of the 12th International Conference on Learning Representations (ICLR 2024) (2024), https://openreview.net/forum?id=VtmBAGCN7o
15. Yang, J., Jimenez, C.E., Wettig, A., et al.: SWE-agent: Agent-computer interfaces enable automated software engineering. In: Advances in Neural Information Processing Systems 37 (NeurIPS 2024). pp. 50528–50652 (2024). https://doi.org/10.52202/079017-1601
16. Wang, Y., Xu, Z., Huang, Y., et al.: DyFlow: Dynamic workflow framework for agentic reasoning. In: Advances in Neural Information Processing Systems 38 (NeurIPS 2025). pp. 174148–174181 (2025), https://proceedings.neurips.cc/paper_files/paper/2025/hash/fe9910d2b03324faeb5371a9658277bb-Abstract-Conference.html
17. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023). pp. 8634–8652 (2023). https://doi.org/10.52202/075280-0377
18. Madaan, A., Tandon, N., Gupta, P., et al.: Self-refine: Iterative refinement with self-feedback. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023). pp. 46534–46594 (2023). https://doi.org/10.52202/075280-2019
19. Wei, J., Wang, X., Schuurmans, D., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems 35 (NeurIPS 2022). pp. 24824–24837 (2022). https://doi.org/10.52202/068431-1800
20. Yao, S., Yu, D., Zhao, J., et al.: Tree of thoughts: Deliberate problem solving with large language models. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023). pp. 11809–11822 (2023). https://doi.org/10.52202/075280-0517
21. Yao, S., Zhao, J., Yu, D., et al.: ReAct: Synergizing reasoning and acting in language models. In: The Eleventh International Conference on Learning Representations (ICLR 2023) (2023), https://openreview.net/forum?id=WE_vluYUL-X
22. Hao, S., Gu, Y., Ma, H., et al.: Reasoning with language model is planning with world model. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. pp. 8154–8173. Association for Computational Linguistics (2023). https://doi.org/10.18653/v1/2023.emnlp-main.507
23. Zhou, A., Yan, K., Shlapentokh-Rothman, M., et al.: Language agent tree search unifies reasoning, acting, and planning in language models. In: Proceedings of the 41st International Conference on Machine Learning. PMLR, vol. 235, pp. 62138–62160 (2024), https://proceedings.mlr.press/v235/zhou24r.html